推荐的重构方案（Electron + React + Python (FastAPI)），我将为您提供一个详细的重构计划。由于这是一个大型的重构项目，我将分步骤进行：

第一阶段：架构设计

1. 技术栈选择

```
Frontend:
- Electron (桌面外壳)
- React 18 (UI 框架)
- TypeScript (类型安全)
- Material-UI / Ant Design (UI 组件)
- React Router (路由管理)

Backend:
- FastAPI (Python Web 框架)
- Uvicorn (ASGI 服务器)
- SQLAlchemy (ORM)
- PostgreSQL/SQLite (数据库)

Infrastructure:
- WebSocket (实时通信)
- RESTful API
- 进程间通信(IPC)
```

2. 目录结构设计

```
network-planning-tool/
├── electron/                    # Electron 主进程
├── frontend/                    # React 前端应用
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── hooks/
│   │   ├── utils/
│   │   └── types/
│   └── package.json
├── backend/                     # FastAPI 后端
│   ├── app/
│   │   ├── api/
│   │   ├── core/
│   │   ├── models/
```

```
│       │       ├─── services/
│       │       └─── utils/
│       └─── requirements.txt
├─── shared/                          # 共享代码
└─── package.json                     # 根 package.json
```

第二阶段：后端重构（FastAPI）

1. 创建 FastAPI 应用结构

backend/app/main.py:
```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.api import router as api_router
from app.core.config import settings

app = FastAPI(
    title="Network Planning Tool API",
    version="1.0.0",
    description="网络规划工具后端 API"
)

# CORS 配置
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "electron://*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 包含 API 路由
app.include_router(api_router, prefix="/api")

@app.get("/")
async def root():
    return {"message": "Network Planning Tool API"}
```

2. 重构许可证管理模块

backend/app/services/license_service.py:

```python
import json
import os
import hashlib
import base64
import hmac
from datetime import datetime
from cryptography.fernet import Fernet
from typing import Tuple, Optional
from pydantic import BaseModel

class LicenseResponse(BaseModel):
    valid: bool
    message: str
    expire_date: Optional[str] = None

class LicenseManager:
    def __init__(self, license_path: str = "license.dat"):
        # 在生产环境中，密钥应从安全配置中读取
        self.secret_key = os.getenv("LICENSE_SECRET_KEY",

"networkplanningtool_default_secret_key_for_encryption").encode()
        self.license_path = license_path

    def validate_license(self) -> LicenseResponse:
        """验证许可证有效性"""
        if not os.path.exists(self.license_path):
            return LicenseResponse(
                valid=False,
                message="许可证文件未找到"
            )

        try:
            # 读取和验证许可证文件
            with open(self.license_path, 'rb') as f:
                encrypted_content = f.read()

            license_package = json.loads(encrypted_content.decode('utf-8'))

            # 验证签名
            encrypted_data = base64.b64decode(license_package['encrypted_data'])
            stored_signature = license_package['signature']

            calculated_signature = hmac.new(
```

```
            self.secret_key,
            encrypted_data,
            hashlib.sha256
        ).hexdigest()

        if not hmac.compare_digest(calculated_signature, stored_signature):
            return LicenseResponse(
                valid=False,
                message="许可证签名验证失败"
            )

        # 解密数据
        derived_key = base64.urlsafe_b64encode(
            hashlib.sha256(self.secret_key).digest()
        )
        fernet = Fernet(derived_key)
        decrypted_data = fernet.decrypt(encrypted_data)
        license_data = json.loads(decrypted_data.decode())

        # 检查有效期
        expire_date = datetime.strptime(
            license_data['expire_date'],
            '%Y-%m-%d'
        )
        current_date = datetime.now()

        if current_date > expire_date:
            return LicenseResponse(
                valid=False,
                message=f"许可证已于 {license_data['expire_date']} 过期",
                expire_date=license_data['expire_date']
            )

        return LicenseResponse(
            valid=True,
            message="许可证有效",
            expire_date=license_data['expire_date']
        )

    except Exception as e:
        return LicenseResponse(
            valid=False,
            message=f"许可证验证失败: {str(e)}"
        )
```

```

### 3. 重构工参更新模块

**backend/app/services/parameter_service.py**:
```python
import pandas as pd
import numpy as np
import zipfile
import glob
import os
import re
import shutil
from datetime import datetime
from typing import Dict, List, Optional, Tuple
from pathlib import Path
from pydantic import BaseModel

class ParameterUpdateRequest(BaseModel):
    full_param_file: str
    baseline_zip: str

class ParameterUpdateResponse(BaseModel):
    success: bool
    message: str
    updated_file: Optional[str] = None

class NetworkParameterService:
    def __init__(self, base_dir: str = "data"):
        self.base_dir = Path(base_dir)
        self.full_param_dir = self.base_dir / "全量工参"
        self.output_dir = self.base_dir / "输出文件"

        # 创建必要的目录
        self.full_param_dir.mkdir(parents=True, exist_ok=True)
        self.output_dir.mkdir(parents=True, exist_ok=True)

    async def update_network_parameters(self, request: ParameterUpdateRequest) -> ParameterUpdateResponse:
        """更新现网工参"""
        try:
            # 查找全量工参文件
            full_param_files = glob.glob(
                str(self.full_param_dir / "ProjectParameter_mongoose*.xlsx")
```
```

```
)

if not full_param_files:
    return ParameterUpdateResponse(
        success=False,
        message="未找到全量工参文件"
    )

# 选择最新的工参文件
full_param_file = self._get_latest_parameter_file(full_param_files)

# 提取压缩包中的工参文件
lte_sdr_files, lte_itbbu_files, nr_files = await self._extract_parameter_files(
    request.baseline_zip
)

# 加载现网工参数据
lte_online_df = await self._load_lte_online_data(lte_sdr_files, lte_itbbu_files)
nr_online_df = await self._load_nr_online_data(nr_files)

# 加载全量工参数据
lte_full_df, nr_full_df = self._load_full_parameters(full_param_file)

# 更新工参数据
updated = False
if lte_online_df is not None and lte_full_df is not None:
    lte_full_df = self._update_lte_parameters(lte_full_df, lte_online_df)
    updated = True

if nr_online_df is not None and nr_full_df is not None:
    nr_full_df = self._update_nr_parameters(nr_full_df, nr_online_df)
    updated = True

if not updated:
    return ParameterUpdateResponse(
        success=False,
        message="没有需要更新的工参数据"
    )

# 保存更新后的工参文件
new_file_path = self._save_updated_parameters(
    full_param_file,
    lte_full_df,
    nr_full_df
```

```
            )

            return ParameterUpdateResponse(
                success=True,
                message="现网工参更新完成",
                updated_file=str(new_file_path)
            )

        except Exception as e:
            return ParameterUpdateResponse(
                success=False,
                message=f"更新全量工参失败: {str(e)}"
            )

    # 原有方法重构为异步版本
    async def _extract_parameter_files(self, zip_path: str):
        """异步提取工参文件"""
        # 使用 asyncio.to_thread 运行阻塞操作
        import asyncio
        return await asyncio.to_thread(self._sync_extract_parameter_files, zip_path)

    def _sync_extract_parameter_files(self, zip_path: str):
        """同步提取工参文件"""
        # 保持原有逻辑...
        pass

    # 其他方法类似重构...
```

4. 创建 API 路由

backend/app/api/parameters.py:
```python
from fastapi import APIRouter, UploadFile, File, Form, BackgroundTasks
from fastapi.responses import FileResponse
from app.services.parameter_service import (
    NetworkParameterService,
    ParameterUpdateRequest,
    ParameterUpdateResponse
)

router = APIRouter(prefix="/parameters", tags=["工参管理"])

@router.post("/update", response_model=ParameterUpdateResponse)
```

```
async def update_parameters(
    background_tasks: BackgroundTasks,
    full_param_file: UploadFile = File(...),
    baseline_zip: UploadFile = File(...)
):
    """更新现网工参"""
    service = NetworkParameterService()

    # 保存上传的文件
    full_param_path = await save_uploaded_file(full_param_file)
    baseline_zip_path = await save_uploaded_file(baseline_zip)

    # 异步执行更新任务
    request = ParameterUpdateRequest(
        full_param_file=full_param_path,
        baseline_zip=baseline_zip_path
    )

    result = await service.update_network_parameters(request)

    # 清理临时文件
    background_tasks.add_task(cleanup_temp_files, [full_param_path, baseline_zip_path])

    return result

@router.get("/download/{filename}")
async def download_file(filename: str):
    """下载文件"""
    file_path = NetworkParameterService().output_dir / filename
    if not file_path.exists():
        raise HTTPException(status_code=404, detail="文件不存在")

    return FileResponse(
        path=file_path,
        filename=filename,
        media_type="application/octet-stream"
    )
```

第三阶段：前端重构（React + TypeScript）

1. 创建 React 应用结构

frontend/package.json:

```json
{
    "name": "network-planning-frontend",
    "version": "1.0.0",
    "private": true,
    "dependencies": {
        "react": "^18.2.0",
        "react-dom": "^18.2.0",
        "@mui/material": "^5.14.0",
        "@mui/icons-material": "^5.14.0",
        "@emotion/react": "^11.11.0",
        "@emotion/styled": "^11.11.0",
        "axios": "^1.5.0",
        "react-router-dom": "^6.15.0",
        "recharts": "^2.8.0",
        "react-dropzone": "^14.2.3",
        "file-saver": "^2.0.5"
    },
    "scripts": {
        "start": "react-scripts start",
        "build": "react-scripts build",
        "test": "react-scripts test",
        "eject": "react-scripts eject"
    },
    "devDependencies": {
        "@types/react": "^18.2.0",
        "@types/react-dom": "^18.2.0",
        "typescript": "^5.1.0",
        "react-scripts": "5.0.1"
    }
}
```

2. 创建 API 服务层

frontend/src/services/api.ts:
```typescript
import axios, { AxiosResponse } from 'axios';

const API_BASE_URL = process.env.REACT_APP_API_URL || 'http://localhost:8000';

export interface ParameterUpdateRequest {
    full_param_file: File;
    baseline_zip: File;
```

```
}

export interface ParameterUpdateResponse {
  success: boolean;
  message: string;
  updated_file?: string;
}

export interface NeighborPlanningRequest {
  cells_file: File;
  params_file: File;
  planning_type: 'NR 到 NR' | 'LTE 到 LTE' | 'NR 到 LTE';
  max_neighbors?: number;
}

export interface PCIPlanningRequest {
  cells_file: File;
  params_file: File;
  network_type: 'LTE' | 'NR';
  reuse_distance_km: number;
  lte_inherit_mod3: boolean;
  nr_inherit_mod30: boolean;
  pci_range?: number[];
}

class NetworkPlanningApi {
  private api = axios.create({
    baseURL: API_BASE_URL,
    timeout: 300000, // 5 分钟超时
  });

  // 工参更新
  async updateParameters(
    request: ParameterUpdateRequest
  ): Promise<ParameterUpdateResponse> {
    const formData = new FormData();
    formData.append('full_param_file', request.full_param_file);
    formData.append('baseline_zip', request.baseline_zip);

    const response = await this.api.post<ParameterUpdateResponse>(
      '/api/parameters/update',
      formData,
      {
        headers: {
```

```
            'Content-Type': 'multipart/form-data',
        },
    }
  );
  return response.data;
}

// 邻区规划
async planNeighbors(
  request: NeighborPlanningRequest
): Promise<AxiosResponse> {
  const formData = new FormData();
  formData.append('cells_file', request.cells_file);
  formData.append('params_file', request.params_file);
  formData.append('planning_type', request.planning_type);
  if (request.max_neighbors) {
    formData.append('max_neighbors', request.max_neighbors.toString());
  }

  return this.api.post('/api/neighbors/plan', formData, {
    headers: {
      'Content-Type': 'multipart/form-data',
    },
    responseType: 'blob',
  });
}

// PCI 规划
async planPCI(request: PCIPlanningRequest): Promise<AxiosResponse> {
  const formData = new FormData();
  formData.append('cells_file', request.cells_file);
  formData.append('params_file', request.params_file);
  formData.append('network_type', request.network_type);
  formData.append('reuse_distance_km', request.reuse_distance_km.toString());
  formData.append('lte_inherit_mod3', request.lte_inherit_mod3.toString());
  formData.append('nr_inherit_mod30', request.nr_inherit_mod30.toString());

  if (request.pci_range) {
    formData.append('pci_range', JSON.stringify(request.pci_range));
  }

  return this.api.post('/api/pci/plan', formData, {
    headers: {
      'Content-Type': 'multipart/form-data',
```

```
        },
        responseType: 'blob',
    });
  }

  // 下载文件
  async downloadFile(filename: string): Promise<Blob> {
    const response = await this.api.get(`/api/files/download/${filename}`, {
        responseType: 'blob',
    });
    return response.data;
  }

  // 检查许可证
  async checkLicense(): Promise<any> {
    return this.api.get('/api/license/check');
  }
}

export const api = new NetworkPlanningApi();
```

3. 创建主界面组件

frontend/src/pages/MainPage.tsx:
```tsx
import React, { useState } from 'react';
import {
  Container,
  Box,
  Paper,
  Typography,
  Tabs,
  Tab,
  LinearProgress,
  Alert,
  Grid,
} from '@mui/material';
import { ParameterUpdatePanel } from '../components/ParameterUpdatePanel';
import { NeighborPlanningPanel } from '../components/NeighborPlanningPanel';
import { PCIPlanningPanel } from '../components/PCIPlanningPanel';
import { LicenseCheck } from '../components/LicenseCheck';

interface TabPanelProps {
```

```
  children?: React.ReactNode;
  index: number;
  value: number;
}

function TabPanel(props: TabPanelProps) {
  const { children, value, index, ...other } = props;

  return (
    <div
      role="tabpanel"
      hidden={value !== index}
      id={`main-tabpanel-${index}`}
      aria-labelledby={`main-tab-${index}`}
      {...other}
    >
      {value === index && <Box sx={{ p: 3 }}>{children}</Box>}
    </div>
  );
}

export const MainPage: React.FC = () => {
  const [tabValue, setTabValue] = useState(0);
  const [isLoading, setIsLoading] = useState(false);
  const [licenseValid, setLicenseValid] = useState<boolean | null>(null);

  const handleTabChange = (event: React.SyntheticEvent, newValue: number) => {
    setTabValue(newValue);
  };

  return (
    <Container maxWidth="xl">
      <Box sx={{ my: 4 }}>
        <Typography variant="h3" component="h1" gutterBottom align="center">
          网络规划工具
        </Typography>
        <Typography variant="subtitle1" align="center" color="text.secondary">
          支持工参更新、邻区规划、PCI 规划等功能
        </Typography>
      </Box>

      <LicenseCheck
        onLicenseChecked={(valid) => setLicenseValid(valid)}
      />
```

```
{licenseValid === false && (
  <Alert severity="error" sx={{ mb: 2 }}>
    许可证验证失败，部分功能可能受限
  </Alert>
)}

<Paper elevation={3} sx={{ width: '100%' }}>
  <Box sx={{ borderBottom: 1, borderColor: 'divider' }}>
    <Tabs
      value={tabValue}
      onChange={handleTabChange}
      aria-label="功能选项卡"
      variant="fullWidth"
    >
      <Tab label="工参更新" />
      <Tab label="邻区规划" />
      <Tab label="PCI 规划" />
    </Tabs>
  </Box>

  {isLoading && <LinearProgress />}

  <TabPanel value={tabValue} index={0}>
    <ParameterUpdatePanel
      onLoadingChange={setIsLoading}
      licenseValid={licenseValid}
    />
  </TabPanel>

  <TabPanel value={tabValue} index={1}>
    <NeighborPlanningPanel
      onLoadingChange={setIsLoading}
      licenseValid={licenseValid}
    />
  </TabPanel>

  <TabPanel value={tabValue} index={2}>
    <PCIPlanningPanel
      onLoadingChange={setIsLoading}
      licenseValid={licenseValid}
    />
  </TabPanel>
</Paper>
```

```
        </Container>
    );
};
```

4. 创建工参更新组件

frontend/src/components/ParameterUpdatePanel.tsx:
```tsx
import React, { useState } from 'react';
import {
    Box,
    Button,
    Card,
    CardContent,
    Grid,
    Typography,
    Alert,
    LinearProgress,
} from '@mui/material';
import { CloudUpload } from '@mui/icons-material';
import { useDropzone } from 'react-dropzone';
import { api, ParameterUpdateResponse } from '../services/api';

interface ParameterUpdatePanelProps {
    onLoadingChange: (loading: boolean) => void;
    licenseValid: boolean | null;
}

export const ParameterUpdatePanel: React.FC<ParameterUpdatePanelProps> = ({
    onLoadingChange,
    licenseValid,
}) => {
    const [fullParamFile, setFullParamFile] = useState<File | null>(null);
    const [baselineZip, setBaselineZip] = useState<File | null>(null);
    const [result, setResult] = useState<ParameterUpdateResponse | null>(null);
    const [error, setError] = useState<string | null>(null);

    const fullParamDropzone = useDropzone({
        accept: {
            'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet': ['.xlsx'],
        },
        onDrop: (acceptedFiles) => {
            if (acceptedFiles.length > 0) {
```

```
            setFullParamFile(acceptedFiles[0]);
        }
    },
});

const baselineDropzone = useDropzone({
    accept: {
        'application/zip': ['.zip'],
    },
    onDrop: (acceptedFiles) => {
        if (acceptedFiles.length > 0) {
            setBaselineZip(acceptedFiles[0]);
        }
    },
});

const handleSubmit = async () => {
    if (!fullParamFile || !baselineZip) {
        setError('请选择需要上传的文件');
        return;
    }

    if (licenseValid === false) {
        setError('许可证验证失败，无法执行操作');
        return;
    }

    setError(null);
    setResult(null);
    onLoadingChange(true);

    try {
        const response = await api.updateParameters({
            full_param_file: fullParamFile,
            baseline_zip: baselineZip,
        });

        setResult(response);

        if (response.success && response.updated_file) {
            // 自动下载更新后的文件
            const blob = await api.downloadFile(response.updated_file);
            const url = window.URL.createObjectURL(blob);
            const a = document.createElement('a');
```

```
            a.href = url;
            a.download = response.updated_file;
            a.click();
        }
    } catch (err: any) {
        setError(err.message || '更新失败');
    } finally {
        onLoadingChange(false);
    }
};

const isDisabled = !fullParamFile || !baselineZip || licenseValid === false;

return (
    <Box>
        <Typography variant="h6" gutterBottom>
            现网工参更新
        </Typography>

        <Grid container spacing={3}>
            <Grid item xs={12} md={6}>
                <Card>
                    <CardContent>
                        <Typography variant="subtitle1" gutterBottom>
                            全量工参文件 (.xlsx)
                        </Typography>
                        <Box
                            {...fullParamDropzone.getRootProps()}
                            sx={{
                                border: '2px dashed #ccc',
                                borderRadius: 1,
                                p: 3,
                                textAlign: 'center',
                                cursor: 'pointer',
                                '&:hover': {
                                    borderColor: 'primary.main',
                                },
                            }}
                        >
                            <input {...fullParamDropzone.getInputProps()} />
                            <CloudUpload sx={{ fontSize: 48, color: 'action.active' }} />
                            <Typography variant="body2" sx={{ mt: 1 }}>
                                {fullParamFile
                                    ? `已选择: ${fullParamFile.name}`
```

```
                : '拖放文件或点击选择'}
            </Typography>
        </Box>
    </CardContent>
  </Card>
</Grid>

<Grid item xs={12} md={6}>
  <Card>
    <CardContent>
      <Typography variant="subtitle1" gutterBottom>
        现网工参压缩包 (.zip)
      </Typography>
      <Box
        {...baselineDropzone.getRootProps()}
        sx={{
          border: '2px dashed #ccc',
          borderRadius: 1,
          p: 3,
          textAlign: 'center',
          cursor: 'pointer',
          '&:hover': {
            borderColor: 'primary.main',
          },
        }}
      >
        <input {...baselineDropzone.getInputProps()} />
        <CloudUpload sx={{ fontSize: 48, color: 'action.active' }} />
        <Typography variant="body2" sx={{ mt: 1 }}>
          {baselineZip
            ? `已选择: ${baselineZip.name}`
            : '拖放文件或点击选择'}
        </Typography>
      </Box>
    </CardContent>
  </Card>
</Grid>

<Grid item xs={12}>
  <Button
    variant="contained"
    color="primary"
    size="large"
    fullWidth
```

```
                onClick={handleSubmit}
                disabled={isDisabled}
            >
                开始更新工参
            </Button>
        </Grid>
    </Grid>

    {error && (
        <Alert severity="error" sx={{ mt: 2 }}>
            {error}
        </Alert>
    )}

    {result && (
        <Alert
            severity={result.success ? 'success' : 'warning'}
            sx={{ mt: 2 }}
        >
            {result.message}
        </Alert>
    )}
  </Box>
);
};
```

第四阶段：Electron 集成

1. Electron 主进程

electron/main.js:
```javascript
const { app, BrowserWindow, ipcMain, session } = require('electron');
const path = require('path');
const { spawn } = require('child_process');
const fs = require('fs');

let mainWindow;
let pythonProcess = null;

function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1400,
```

```
        height: 900,
        webPreferences: {
            nodeIntegration: false,
            contextIsolation: true,
            preload: path.join(__dirname, 'preload.js'),
        },
        icon: path.join(__dirname, 'assets/icon.png'),
    });

    // 加载 React 应用
    if (process.env.NODE_ENV === 'development') {
        mainWindow.loadURL('http://localhost:3000');
        mainWindow.webContents.openDevTools();
    } else {
        mainWindow.loadFile(path.join(__dirname, '../frontend/build/index.html'));
    }

    mainWindow.on('closed', () => {
        mainWindow = null;
    });
}

// 启动 Python 后端
function startBackend() {
    const backendPath = path.join(__dirname, '../backend');

    pythonProcess = spawn('python', ['-m', 'uvicorn', 'app.main:app', '--host', '127.0.0.1', '--port',
'8000'], {
        cwd: backendPath,
        stdio: ['pipe', 'pipe', 'pipe']
    });

    pythonProcess.stdout.on('data', (data) => {
        console.log(`Python 后端: ${data}`);
    });

    pythonProcess.stderr.on('data', (data) => {
        console.error(`Python 后端错误: ${data}`);
    });

    pythonProcess.on('close', (code) => {
        console.log(`Python 后端进程退出，退出码: ${code}`);
    });
}
```

```
// IPC 通信处理
ipcMain.handle('get-app-path', () => {
    return app.getPath('userData');
});

ipcMain.handle('read-license', async () => {
    const licensePath = path.join(app.getPath('userData'), 'license.dat');
    if (fs.existsSync(licensePath)) {
        return fs.readFileSync(licensePath, 'utf-8');
    }
    return null;
});

ipcMain.handle('save-license', async (event, licenseData) => {
    const licensePath = path.join(app.getPath('userData'), 'license.dat');
    fs.writeFileSync(licensePath, licenseData);
    return true;
});

app.whenReady().then(() => {
    createWindow();
    startBackend();

    app.on('activate', () => {
        if (BrowserWindow.getAllWindows().length === 0) {
            createWindow();
        }
    });
});

app.on('window-all-closed', () => {
    if (process.platform !== 'darwin') {
        app.quit();
    }
});

app.on('before-quit', () => {
    if (pythonProcess) {
        pythonProcess.kill();
    }
});
```

2. Electron 预加载脚本

electron/preload.js:
```javascript
const { contextBridge, ipcRenderer } = require('electron');

// 暴露安全的 API 给渲染进程
contextBridge.exposeInMainWorld('electronAPI', {
  getAppPath: () => ipcRenderer.invoke('get-app-path'),
  readLicense: () => ipcRenderer.invoke('read-license'),
  saveLicense: (licenseData) => ipcRenderer.invoke('save-license', licenseData),

  // 文件系统操作
  readFile: (filePath) => ipcRenderer.invoke('read-file', filePath),
  writeFile: (filePath, content) => ipcRenderer.invoke('write-file', filePath, content),

  // 通知
  showNotification: (title, body) => ipcRenderer.invoke('show-notification', title, body),
});
```

3. 根 package.json 配置

package.json:
```json
{
  "name": "network-planning-tool",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev:frontend": "cd frontend && npm start",
    "dev:backend": "cd backend && uvicorn app.main:app --reload",
    "dev": "concurrently \"npm run dev:frontend\" \"npm run dev:backend\"",
    "build:frontend": "cd frontend && npm run build",
    "build:electron": "electron-builder",
    "build": "npm run build:frontend && npm run build:electron",
    "start": "electron .",
    "package": "npm run build:frontend && electron-builder --dir",
    "dist": "npm run build:frontend && electron-builder"
  },
  "devDependencies": {
    "concurrently": "^8.2.0",
    "electron": "^25.0.0",
    "electron-builder": "^24.0.0"
```

```
  },
  "build": {
    "appId": "com.networkplanning.tool",
    "productName": "网络规划工具",
    "directories": {
      "output": "dist"
    },
    "files": [
      "electron/**/*",
      "frontend/build/**/*",
      "backend/**/*"
    ],
    "extraResources": [
      {
        "from": "backend",
        "to": "backend",
        "filter": ["**/*"]
      }
    ],
    "win": {
      "target": "nsis",
      "icon": "assets/icon.ico"
    },
    "mac": {
      "target": "dmg",
      "icon": "assets/icon.icns"
    },
    "linux": {
      "target": "AppImage",
      "icon": "assets/icon.png"
    }
  }
}
```

第五阶段：重构注意事项和保留功能

1. 核心功能保留
- ☑ 许可证验证机制
- ☑ 工参更新功能
- ☑ 邻区规划功能（NR 到 NR、LTE 到 LTE、NR 到 LTE）
- ☑ PCI 规划功能（LTE 和 NR 分离式）
- ☑ 文件导出功能
- ☑ 配置参数保留

2. 改进的功能
- 异步文件处理，支持大文件
- 进度提示和错误处理
- 用户友好的界面
- 多平台支持（Windows、macOS、Linux）
- 更好的性能和内存管理

3. 移除的功能
- ❌ Tkinter GUI 框架
- ❌ 同步阻塞的 UI 操作
- ❌ 平台特定的文件对话框

4. 部署配置

backend/requirements.txt:
```txt
fastapi==0.104.0
uvicorn[standard]==0.24.0
pandas==2.1.1
numpy==1.24.3
openpyxl==3.1.2
geopandas==0.14.0
cryptography==41.0.3
python-multipart==0.0.6
sqlalchemy==2.0.20
alembic==1.12.0
python-jose[cryptography]==3.3.0
python-dotenv==1.0.0
```

总结

这个重构方案提供了：

1. **现代化架构**：前后端分离，易于维护和扩展
2. **更好的用户体验**：React 提供响应式、现代化的 UI
3. **跨平台支持**：Electron 支持 Windows、macOS、Linux
4. **性能改进**：异步处理，避免 UI 阻塞
5. **可扩展性**：易于添加新功能和模块
6. **类型安全**：TypeScript 减少运行时错误
7. **更好的错误处理**：完善的错误处理和用户反馈

重构需要分阶段进行，建议：

1. 先实现后端 API
2. 再实现 React 前端
3. 最后集成 Electron
4. 逐步迁移原有功能，确保每一步都能正常工作